CRYPTANALYSIS OF LATTICE-BASED SCHEME USING HEURISTIC-BASED OPTIMIZATION TECHNIQUES

A PROJECT REPORT

submitted by

Rohan Kalra 21114083

Shrey Gupta 21112103

Tanmay Mohit Shrivastav 21114103

for the fulfillment
of
CSN-400B: B.Tech. Project

under the guidance of
Prof. Sugata Gangopadhyay

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY ROORKEE
ROORKEE - 247667
2024 - 2025

CANDIDATE'S DECLARATION

We hereby declare that the work carried out in the B.Tech. Project titled **"CRYPTANALYSIS OF LATTICE-BASED SCHEME USING HEURISTIC-BASED OPTIMIZATION TECHNIQUES"** is presented for the partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in the Computer Science and Engineering and submitted to the Department of Computer Science and Engineering, Indian Institute of Technology Roorkee under the supervision of Prof. Sugata Gangopadhyay , Department of CSE, IIT Roorkee.

The work presented in this report is an authentic record of our work carried out during the period from July 2024 to June 2025. The content of this report has not been submitted by us for the award of any other degree of this or any other institute.

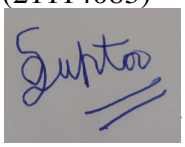
Place : IIT Roorkee

Date : May 23, 2025

Signature of student : .....

Name of student : Rohan Kalra

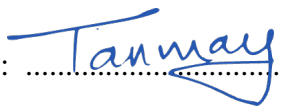
: (21114083)



Signature of student : .....

Name of student : Shrey Gupta

: (21112103)



Signature of student : .....

Name of student : Tanmay Mohit Shrivastav

: (21114103)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

ROORKEE - 247667

2024 - 2025

CERTIFICATE

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Place : IIT Roorkee

Date : May 23, 2025

Signature of the Supervisor : .....

Name of the Supervisor : Prof. Sugata Gangopadhyay

ACKNOWLEDGMENT

We feel extremely indebted to everyone who helped our team work on the Project titled **Cryptanalysis of Lattice-based scheme using Heuristic-based optimization techniques** and present it.

Foremost, we would especially thank our supervisor, Prof. Sugata Gangopadhyay from the Department of COMPUTER SCIENCE & ENGINEERING for giving us valuable suggestions and critical inputs in the preparation of this report.

We are also extremely grateful to Mr. Ali Raya for his constant support. His guidance has been instrumental in allowing us to move forward with our endeavor.

We send our heartfelt appreciation and gratitude to our families and friends, who have supported us throughout our journey. It is them whose love and constant encouragement have motivated us throughout our journey.

In the end, we would thank all those whose names may not have been mentioned here but have had a deep impact on our lives.

ABSTRACT

The objective of post-quantum cryptography (also referred to as quantum-resistant cryptography) is to design cryptographic protocols that remain secure against adversaries equipped with both classical and quantum computational capabilities. Several approaches fall under the umbrella of post-quantum cryptography, including lattice-based, code-based, multivariate polynomial-based, hash-based, and isogeny-based schemes. Among these, lattice-based cryptographic systems are the most extensively studied and standardized, primarily due to their foundation in hard mathematical problems such as the Shortest Vector Problem (SVP).

The aim of this project is to implement a genetic algorithm tailored to solve the SVP for lattices up to a certain dimension within a practical time frame. Genetic algorithms, which are part of a larger class of evolutionary algorithms, are adaptive heuristic search techniques inspired by the principles of natural selection and genetics. They are widely used to address complex optimization and search problems.

In this work, we explore the use of genetic algorithms to efficiently find short vectors in lattice structures using limited computational resources. Additionally, we examine the potential application of these methods in constructing and analyzing lattice-based cryptographic systems. The entire implementation has been carried out in `C++`.

CONTENTS

ACKNOWLEDGMENT i

ABSTRACT ii

LIST OF TABLES v

LIST OF FIGURES vi

Chapter 1. INTRODUCTION 1

1.1 Motivation 1

1.2 Objective 2

1.3 Problem Statement 2

1.3.1 Shortest Vector Problem (SVP): 3

1.3.2 Closest Vector Problem (CVP): 3

1.4 Applications 3

Chapter 2. LITERATURE SURVEY 4

2.0.1 Paper 1- An Accelerated Algorithm for Solving SVP Based on Statistical Analysis 4

Chapter 3. PRELIMINARIES 11

3.1 Genetic Algorithm 11

3.2 Gram-Schmidt Orthogonalization 13

3.3 Blockwise Korkin-Zolotarev Basis 13

3.4 Sieving 14

3.4.1 Comparison of GA and Sieving Methods . . . 15

3.5 Y-spare Representation 16

3.5.1 Introduction 17

3.5.2 Initialization 18
3.5.3 Selection 19
3.5.4 Crossover 19
3.5.5 Mutation 20
3.5.6 Encoding and Decoding 20
3.5.7 Local Search 21
3.5.8 Heuristic Pruning 21
3.6 Natural Number Representation 22
3.6.1 Introduction 22
3.6.2 Initialization 23
3.6.3 Crossover 24
3.6.4 Mutation 25
3.6.5 Fitness Evaluation Function 26
3.6.6 Elitist Selection Strategy 27
3.6.7 Encoding and Decoding 28
3.7 Comparison: Y-Sparse vs NNR 30

Chapter 4. METHODOLOGY **33**
4.1 Preprocess class 34
4.2 Individual Class 34
4.3 Repres Class 35
4.4 GA class 36
4.5 NTRU class 37

Chapter 5. UNIQUE ASPECTS OF THE PROJECT **38**
5.1 Appication of OOPs 38
5.2 Use of `fplll` Library for Efficient Lattice Computations . . 39
5.2.1 Key Features of fplll 40
5.2.2 Documentation Source 41
5.3 Enhancing Flexibility with templates 41
5.3.1 Template-based Classes in Our Project 41

5.4 Memory Management with Smart Pointers 42

Chapter 6. RESULTS & DISCUSSION **44**

Chapter 7. FUTURE WORKS **47**

LIST OF TABLES

6.1 Comparison of GA with Y sparse + Local Search and BKZ Algorithm (Beta = 40) Across Different Dimensions. 44
6.2 Comparison of GA with NNR + Local Search and BKZ Algorithm (Beta = 40) Across Different Dimensions. 45

LIST OF FIGURES

3.1 Flow of Genetic Algorithm 12

4.1 Class Diagram 33

5.1 Example Usage of OOPs 38
5.2 FPLLL Structure 41
5.3 Memory usage without Smart Pointers 43
5.4 Optimized memory usage 43

6.1 Comparison of Y-sparse and NNR 46

Chapter 1

INTRODUCTION

1.1 MOTIVATION

There has been a substantial progress in the field of quantum computers that are used to solve complex mathematical problems by using quantum mechanical phenomenon. Emergence of large scale quantum computers could potentially break many of the public-key crypto-systems that are currently in use. Such systems are predicted to be constructed in the future.This poses a significant threat to integrity and confidentiality of digital communications on the internet and elsewhere.Thus research in post quantum cryptography now is necessary to avoid widespread chaos in the future. NIST has started a project to standardise PQC schemes and the only key encapsulation mechansism they have standardised is Kyber Bos et al. (2018) which is lattice based also two out of three signature schemes that have been standardised are lattice based. This shows how important lattice based cryptography is and why it needs to be tested rigorously. A rudimentary challenge in lattice-based cryptographic schemes is the Shortest Vector Problem (SVP), which involves identifying the shortest non-zero vector that lies in a lattice generated by a given basis. This problem is classified as NP-hard and is known to require exponential time to solve using existing algorithms.

1.2 OBJECTIVE

The aim of this project is to design an algorithm that addresses the Shortest Vector Problem (SVP) through evolutionary search methods. Specifically, it focuses on identifying the shortest non-zero vector that lies within the span of a given lattice basis. Traditional approaches to solving SVP are known to have exponential time complexity relative to the input size. This work seeks to evaluate the efficiency of evolutionary techniques in solving SVP and examine their potential in robustly analyzing the security of lattice-based cryptographic systems.

1.3 PROBLEM STATEMENT

A lattice L is the collection of all integer linear combinations of n linearly independent vectors $b_1, b_2, \ldots, b_n \in \mathbb{Z}^m$, which together form the basis of the lattice. The rank of the lattice is n, and we typically consider full-rank lattices where the dimension m is equal to the rank n.

A lattice L is defined over a matrix $B = [b_1, b_2, \ldots, b_n]$ whose columns are linearly independent vectors, as follows:

$$L = \{x_1 b_1 + x_2 b_2 + \cdots + x_n b_n \mid x_1, x_2, \ldots, x_n \in \mathbb{Z}\} \tag{1.1}$$

The fundamental domain F of the lattice with respect to the basis vectors is given by:

$$F(b_1, \ldots, b_n) = \{x_1 b_1 + x_2 b_2 + \cdots + x_n b_n \mid 0 \leq x_i < 1\} \tag{1.2}$$

The Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP) are two fundamental computational problems that lie at the core of lattice-based cryptography.

1.3.1 Shortest Vector Problem (SVP):

This problem focuses on locating the shortest non-zero vector within a lattice L. Specifically, it requires finding a vector $v \in L$ such that its Euclidean length $\|v\|$ is as small as possible.

1.3.2 Closest Vector Problem (CVP):

Given a target vector w that lies outside the lattice L, the objective is to determine a lattice vector $v \in L$ that is nearest to w, minimizing the distance $\|w - v\|$ in Euclidean space.

1.4 APPLICATIONS

We believe that genetic algorithms can prove to be good tools for solving hard lattice problems like SVP. In this project, we explore a similar approach for genetic algorithm using y-sparse representation which has proven to be effective in solving the SVP in an acceptable time. The y-sparse representation provides a way to find a sparse integer representation for lattice vectors and then encode them into bit strings. It also provides a complete genetic algorithm that has been proven to converge using Markov chain analysis. There are also some improvements like local search and heuristic pruning. We have implemented this code in c++ and have tried to make the code as modular using Object-Oriented Programming as possible. We also provide documentation of the code and class diagram for better understandability of the code. Using OOP, we can also make the code more reusable and provide room for more optimizations in the future. We have also tested the code on several SVP-challenge lattices and provided results for the same.

Chapter 2

LITERATURE SURVEY

In this section, we provide an overview of the existing research and development regarding genetic algorithm and the shortest vector problem. We will explain relevant theories related to our field which we have used extensively in our own experiments.The following are the two papers we have used as reference that form the foundation of our own implementation.

2.0.1 Paper 1- An Accelerated Algorithm for Solving SVP Based on Statistical Analysis

Authors: Masaharu Fukase, Kenji Kashiwabara

[3] **Summary**

This paper presents an algorithm that serves as a core component of the Random Sampling Reduction (RSR) technique, a method employed to find short lattice vectors for solving the Shortest Vector Problem (SVP).

Overview of the Algorithm

The Sampling Algorithm (SA) generates a lattice vector $v \in L(B)$ by constructing a linear combination of the Gram-Schmidt orthogonalized basis vectors $b_1^*, b_2^*, \ldots, b_n^*$. Any lattice vector $v \in L(B)$ can be written as:

$$v = \sum_{j=1}^{n} \nu_j b_j^*$$

where $\nu_j \in \mathbb{R}$ are the Gram-Schmidt coefficients determined by a specific linear combination of the original basis vectors.

For a reduced basis, it is typically observed that the Gram-Schmidt vectors decrease in norm:

$$\|b_1^*\| > \|b_2^*\| > \cdots > \|b_n^*\|$$

This property implies that early (low-index) basis vectors contribute more significantly to the overall Euclidean norm of v.

Therefore, in order to generate a short lattice vector, the corresponding coefficients v_j for small j must be small in magnitude. Conversely, slightly larger coefficients are tolerable for higher-index (and hence smaller norm) vectors b_j^*.

The Sampling Algorithm enforces this by restricting the magnitude of v_j based on the index j:

- For small j, sample v_j such that $|v_j| \leq \frac{1}{2}$
- For larger j, allow $|v_j| \leq 1$
- Fix $v_n = 1$ to ensure the vector is non-zero

This selective sampling increases the probability of generating a short vector, making the process statistically efficient.

Input:

- Lattice basis $B = [\mathbf{b}_1, \ldots, \mathbf{b}_n]$
- An integer u such that $1 \leq u < n$

Output:

- A lattice vector $\mathbf{v}$ satisfying Eq. (1)

Steps:

1. Compute the Gram-Schmidt coefficients $\mu_{i,j}$ such that:

$$\mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\langle \mathbf{b}_j^*, \mathbf{b}_j^* \rangle}$$

where $\mathbf{b}_j^*$ denotes the j-th Gram-Schmidt vector.

2. Initialize $\mathbf{v} := \mathbf{b}_n$

3. For $j = 1$ to $n-1$, set:

$$\mu_j := \mu_{n,j}$$

4. For $i = n-1$ down to 1, do:

   (a) Select $y \in \mathbb{Z}$ uniformly at random such that:

$$|\mu_i - y| \leq \begin{cases} \frac{1}{2} & \text{if } i < n-u \\ 1 & \text{if } i \geq n-u \end{cases}$$

   (b) Update the vector:

$$\mathbf{v} := \mathbf{v} - y \cdot \mathbf{b}_i$$

   (c) For $j = 1$ to $i-1$, update:

$$\mu_j := \mu_j - y \cdot \mu_{i,j}$$

Return: the lattice vector $\mathbf{v}$

Why the Algorithm Works

To understand why the Sampling Algorithm (SA) is effective at generating short lattice vectors, we must analyze how a vector present in the lattice can be represented in the Gram-Schmidt orthogonalized basis.

Let $B = [b_1, b_2, \ldots, b_n]$ be a set of lattice basis, and let b_j^* denote the Gram-Schmidt orthogonalized vectors. Each basis vector b_i can be written as a linear combination of the GSO vectors:

$$b_i = \sum_{j=1}^{i} \mu_{i,j} b_j^* \tag{2.1}$$

Now, consider a lattice vector $v \in L(B)$, which can be expressed in terms of the basis B as:

$$v = \sum_{i=1}^{n} x_i b_i \tag{2.2}$$

where $x_i \in \mathbb{Z}$.

Substituting the expression for b_i from Equation (2.1) into Equation (2.2), we obtain:

$$\begin{aligned} v &= \sum_{i=1}^{n} x_i \left(\sum_{j=1}^{i} \mu_{i,j} b_j^* \right) \\ &= \sum_{j=1}^{n} \left(\sum_{i=j}^{n} x_i \mu_{i,j} \right) b_j^* \end{aligned} \tag{2.3}$$

This gives us a new representation of the lattice vector v as a linear combination of the orthogonalized basis vectors b_j^*, where the coefficient for each b_j^* is:

$$v_j = \sum_{i=j}^{n} x_i \mu_{i,j}$$

Now, since $\mu_{j,j} = 1$, we can separate the direct contribution from x_j as follows:

$$v_j = x_j + \sum_{i=j+1}^{n} x_i \mu_{i,j} \tag{2.4}$$

This shows that each v_j depends on the current coefficient x_j and a residual sum involving future x_i terms weighted by $\mu_{i,j}$.

In the Sampling Algorithm, we work backward from $j = n$ to $j = 1$, keeping track of the accumulated residual sum

$$\mu_j = \sum_{i=j+1}^{n} x_i \mu_{i,j}$$

and then selecting x_j such that $v_j = x_j + \mu_j$ lies within a bounded interval (like $[-\frac{1}{2}, \frac{1}{2}]$ or $[-1, 1]$).

This formulation justifies the rounding strategy used in the algorithm and shows how careful control of x_j ensures v_j remains within the desired range to help produce a short lattice vector.

Relevance to This Project:

The Sampling Algorithm is a key building block in generating high-quality short lattice vector candidates based on statistical assumptions and controlled coefficient sampling. In our genetic algorithm framework for solving the Shortest Vector Problem (SVP), it serves as an effective method for initializing the population with vectors that are statistically more likely to be close to the shortest vector.By integrating the Sampling Algorithm into the initialization phase, we ensure that the search process begins in a promising region of the solution space.

Paper 2 - A Genetic Algorithm for Searching Shortest Lattice Vector of SVP Challenge

Authors: Dan Ding, Guizhen Zhu, Xiaoyun Wang

[1] **Summary**

This paper presents a novel genetic algorithm (GA) designed to solve the Shortest Vector Problem (SVP) in lattice theory, an essential problem in cryptography. We primarily use sparse integer representations of numbers in order to form our chromosome to ensure efficient encoding and decoding of vectors. The key contributions include:

1. **Sparse Integer Representations:** The paper introduces the Y-sparse representation to represent lattice vectors which uses the Gram-Schmidt Orthogonalization to improve encoding.

2. **Heuristic Enhancements:** Optimizations like Local Search and Heuristic Pruning are introduced in this paper.

4. **Experimental Results:** The GA is tested against SVP challenges, outperforming

traditional algorithms like Kannan-Helfrich enumeration and conservative pruning.

Relevance to This Project:

This paper forms the foundation for our implementation of the genetic algorithm. We use their novel way of representing intergers in our algorithm as well.

Paper 3 - A Genetic Algorithm with Restart Strategy for Solving Approximate Shortest Vector Problem

[4] **Summary:**

This paper introduces a genetic algorithm with a restart strategy to solve the Shortest Vector Problem. The key contributions are outlined below:

1. **Integer Representation:** The authors propose an efficient representation of lattice vectors using the natural number representation(NNR).

2. **Algorithm Design:** The GA incorporates a restart strategy to avoid premature convergence and enhance exploration.

3. **Optimizations:** Optimizations include local search mechanisms and adjustments to the mutation strategy.

4. **Experimental Results:** The proposed GA is tested on ASVP benchmarks, demonstrating significant improvements in efficiency and solution quality compared to traditional methods.

Relevance to This Project:

The integer representation and optimization techniques from this paper provide a strong foundation for adapting genetic algorithms to the Shortest Vector Problem (SVP). The restart strategy and heuristic improvements are particularly relevant for enhancing the performance of GA-based approaches in cryptographic applications.

Paper 4 - Improving Genetic Algorithms for Solving the SVP: Focusing on Low Memory Consumption and High Reproducibility

Authors: Masaharu Fukase and Masahiro Kaminaga

[2] **Summary**

This paper mainly focuses on creating a genetic algorithm that is efficient, consumes less memory and has high reproducibility.They have presented new interpretations to crossover and mutation.The adaptive design of the algorithm achieves a better balance in exploring the solution space and moving towards better solutions. Given below are some of its key features:-

1. **Enhanced Genetic Algorithm:** The paper introduces a genetic algorithm with improved runtime and low memory consumption.Their proposed GA requires only a few Mega Bytes of memory making it suitable for high dimensional lattices.

2. **Optimizations:** The paper provides scope for GPU based parallelization to further enhance the algorithm's scalability.

Chapter 3

PRELIMINARIES

3.1 GENETIC ALGORITHM

Genetic algorithm is an algorithm that takes inspiration from processes of natural selection and genetics. This algorithm genre is gaining popularity because it can be richly applied to optimization and search issues. In this algorithm we build up a population of individuals and this population makes crossovers and mutation and undergoes natural selection to generate solutions. This algorithm turns out useful in regimes where traditional algorithms might fail and also a reduced time and space is expected for this algorithm. Following is a brief overview of steps carried out in our genetic algorithm.

Initialization: This step involves instantiating the population's candidates to carry out the genetic algorithm.

Crossover: The exchange of information between two chromosomes/individuals to generate the next individual.

Mutation: A rare event that introduces changes in the information carried by the individual, which helps diversify the population.

Selection: This step involves selecting viable candidates based on the chosen fitness function.

Termination: The search or generation process terminates when the result passes the benchmark threshold.

Optimizations: The search space can be optimized by pruning less viable candidates and considering the local neighbors of viable candidates.

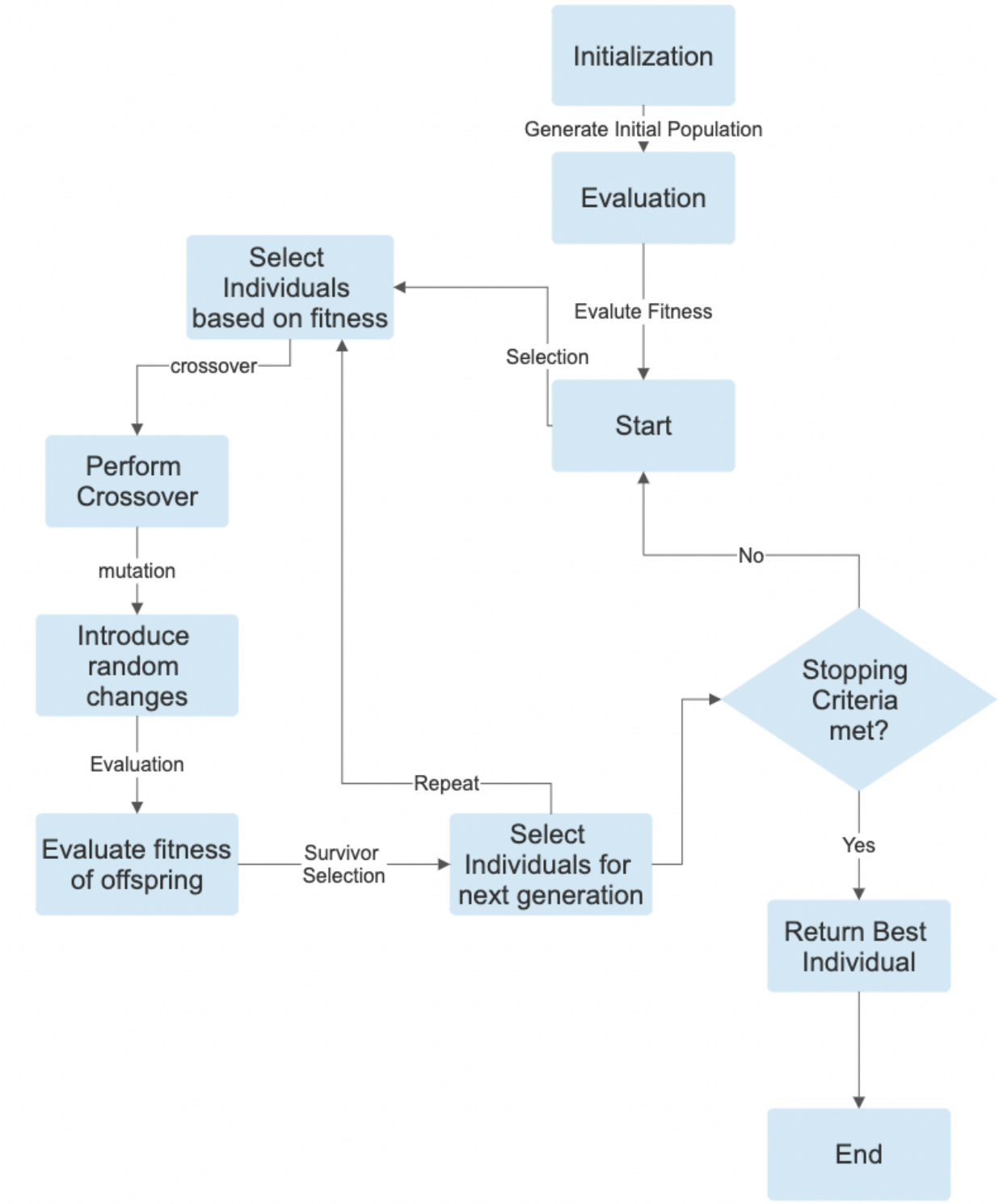


Figure 3.1: Flow of Genetic Algorithm

3.2 GRAM-SCHMIDT ORTHOGONALIZATION

Lattices in this project are considered in the form:

$$L(B) = \{Bx \mid x \in \mathbb{Z}^n\} = \left\{ v \in \mathbb{R}^n \;\middle|\; v = \sum_{i=1}^{n} x_i b_i, x_i \in \mathbb{Z} \right\} \tag{3.1}$$

It is important to note that a lattice can have multiple valid bases; in other words, its representation through basis vectors is not unique. Given a specific basis matrix B, the corresponding parallelepiped $P(B)$ is described as:

$$P(B) = \{Bx \mid x = (x_1, \ldots, x_n) \in \mathbb{R}^n,\ 0 \leq x_i < 1 \text{ for } i = 1, 2, \ldots, n\} \tag{3.2}$$

For a set of basis $B = [b_1, b_2, \ldots, b_n] \in \mathbb{R}^{n \times n}$, the Gram-Schmidt orthogonalization produces an orthogonal set of basis $B^* = [b_1^*, b_2^*, \ldots, b_n^*]$ where each vector is computed iteratively as:

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{ij} b_j^* \tag{3.3}$$

with the projection coefficients μ_{ij} defined as:

$$\mu_{ij} = \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}, \quad \text{for } 1 \leq j < i \leq n \tag{3.4}$$

This process helps decompose the basis into orthogonal components, which is instrumental in various lattice algorithms such as LLL and BKZ.

3.3 BLOCKWISE KORKIN-ZOLOTAREV BASIS

Let $B = [b_1, b_2, \ldots, b_n]$ represent a basis for a lattice $L \subset \mathbb{R}^n$. For each $i \in \{1, 2, \ldots, n\}$, define a projection π_i onto the orthogonal complement of the span of the first $i-1$ basis vectors. This projection maps vectors in $\mathbb{R}^n$ onto the subspace orthogonal to $\text{span}(b_1, b_2, \ldots, b_{i-1})$.

For indices $j > i$, the projection $\pi_i(b_j)$ can be expressed in terms of the Gram-Schmidt orthogonal vectors b_j^* as:

$$\pi_i(b_j) = b_j^* + \sum_{k=i}^{j-1} \mu_{jk} b_k^*, \quad \text{where } i < j \tag{3.5}$$

Notably, this projection satisfies $\pi_i(b_i) = b_i^*$ and $\pi_i(b_j) = 0$ whenever $j < i$.

We define the sublattice $L(k)$ of rank k as the lattice generated by the set $[\pi_i(b_i), \ldots, \pi_i(b_{i+k-1})]$, where the condition $i+k \leq n+1$ ensures valid indices. As a special case, the projected lattice $L_i^{n-i+1} = \pi_i(L)$ has rank $n-i+1$ and is generated by the vectors $[\pi_i(b_i), \ldots, \pi_i(b_n)]$.

A basis B is considered **β-BKZ-reduced** (Blockwise Korkin-Zolotarev reduced with block size β) if the following two conditions are met:

- The Gram-Schmidt coefficients satisfy the size reduction constraint $|\mu_{ij}| \leq \frac{1}{2}$ for all $1 \leq j < i \leq n$.
- For each index i, the vector $\pi_i(b_i)$ is the shortest in the projected sublattice $L_{\min(\beta, n-i+1)}$, as measured by the Euclidean norm.

3.4 SIEVING

Sieving methods provide a more systematic approach to solving SVP. The basic idea is to sample many lattice vectors and iteratively reduce them to obtain shorter ones. Common sieving techniques include:

- **Gauss Sieve:** Iteratively reduces pairs of vectors by applying pairwise reductions.
- **Randomized Sieving:** Utilizes probabilistic techniques to efficiently eliminate long vectors.

- **BKZ and LLL Preprocessing:** Preprocessing the basis can significantly improve sieving performance.

Sieving can also be visualized as an evolutionary algorithm as described in the algorithm below.

Input: A description (basis) of a lattice $\mathscr{L}$

Output: A population $P \subseteq \mathscr{L}$ of short lattice vectors

1. Initialize a population $P \subseteq \mathscr{L}$ of random lattice vectors.
2. **Repeat**

   label=. For all potential parents $v, w \in P$:

   label=. Generate the (potential) offspring $u = v - w$.

   lbbel=. If $\|u\| < \|v\|$ or $\|u\| < \|w\|$, then:

   label=0. Replace the longest parent with u.

3. **Until** P contains sufficiently short lattice vectors.
4. **Return** P.

3.4.1 Comparison of GA and Sieving Methods

3.4.1.(i) **Similarities**

- Both approaches aim to find a short vector in a lattice.
- Both involve iterative improvement of candidate solutions.
- Both can leverage preprocessing techniques such as basis reduction.
- Lattice sieving can be intuitively interpreted as an evolutionary algorithm because its fundamental process involves creating a large population of vectors

and applying simple, local recombination operations on vector pairs to produce shorter vectors, which then replace the longer ones in the population.

3.4.1.(ii) **Differences**

- Sieving methods guarantee finding the shortest vector, while GA provides approximate solutions.
- Sieving methods often have exponential memory requirements, whereas GA is more memory-efficient but slower in convergence.
- GA is a heuristic method with stochastic behavior, while sieving is a deterministic approach with predictable performance.
- GA can handle larger problem instances with reasonable accuracy, while sieving is more constrained by memory limitations.
- There is no concept of Mutation in sieving algorithms.
- In sieving, replacement occurs locally—parents are immediately substituted by their offspring if the offspring are shorter—rather than through a global survivor selection mechanism that evaluates the entire population at once.

3.5 Y-SPARE REPRESENTATION

We have gone with the approach of Y-sparse representation for our implementation purposes, here we make the represen- tation of the vector into a sparse pattern. The approach sets majority of it's elements to 0 and the remaining have a modest non-zero magnitude.

Sparse representation is the indicative of the same, standing for mostly zeroes and rest as non-zero but small values. This representation open gateways for more efficient encoding and faster manipulation and changes to the data stored in the vector-

ized data.

We denote the Sparse representation as follows : $y = [y_1, y_2, y_3 ..... y_n]$

3.5.1 Introduction

In the context of lattice basis reduction, particularly after applying the LLL (Lenstra-Lenstra-Lovász) algorithm, an important property is observed in the shortest vectors: these vectors tend to be *Y-sparse*. This means that among the coordinates of the shortest vector, only a small subset of elements are non-zero, and these non-zero entries usually have relatively small magnitudes.

This sparsity and small magnitude offer several practical advantages:

- **Simplified Representation:** Since most elements are zero, storing and manipulating such vectors becomes computationally efficient, requiring less memory and fewer operations.
- **Reduced Search Space:** The search for the shortest vector can focus on a smaller subset of candidates, as many vector coordinates are zero and the non-zero entries are bounded in size. This reduction decreases the combinatorial complexity of the problem.
- **Efficient Computation:** Algorithms operating on these vectors—such as enumeration or heuristic search methods—can exploit the sparsity to prune the search space and avoid unnecessary calculations involving zero entries.
- **Improved Algorithmic Performance:** Since the non-zero elements are small in magnitude, arithmetic operations like additions and multiplications are less costly, accelerating vector reduction and enumeration procedures.

In summary, the Y-sparsity of shortest vectors in an LLL-reduced basis not only reflects inherent lattice structure but also facilitates more efficient computational methods for identifying these vectors, making the Shortest Vector Problem

more tractable in practice.

We will now describe the working of the genetic algorithm on this representation.

3.5.2 Initialization

The genetic algorithm begins by initializing a population of Y-sparse vectors. Each individual in the population is encoded using the Y-sparse representation, where most elements are zero except for a few non-zero components.

$$\alpha_i = \frac{\|b_1^*\|}{\|b_i^*\|}, \quad \text{for } 1 \leq i \leq n$$

$$l_i = 2 + \log(\alpha_i), \quad \text{for } 1 \leq i \leq n$$

For $k \leftarrow 1$ **to** p **do:**

1. Select integers y_i independently and uniformly at random such that $|y_i| \leq \alpha_i$, for each $i = 1, 2, \ldots, n$.

2. For each i, compute

$$x_i = y_i - \lfloor \tau_i \rfloor,$$

where

$$\tau_i = \sum_{j=i+1}^{n} \mu_{ji} x_j.$$

3. Form the vectors

$$x_k = (x_1, x_2, \ldots, x_n), \quad y_k = (y_1, y_2, \ldots, y_n).$$

4. Store the k-th tuple (x_k, y_k) for further use.

Let's now explore how the algorithm operates on this representation.

3.5.3 Selection

During the selection phase, individuals are chosen from the population based on their fitness for reproduction. We have used an approach of selecting k random individuals from the sample and selecting the best individual out of it based on it's fitness value. We have used a tournament selection approach where we first select random vectors from the population and out of them select 2 vectors based on their norm.

3.5.4 Crossover

Crossover involves combining genetic information from two parent Y-sparse vectors to produce offspring. This step ensures that the resultant has features of individuals from both the parents.

Input: Two Y-sparse vectors in bitstring representation $x = (x_1, \ldots, x_n)$ and $y = (y_1, \ldots, y_n)$.

Output: A new Y-sparse vector in bitstring representation z.

1. Initialize an empty Y-sparse vector z.
2. For $i \leftarrow 1$ to n, do:
   (a) Choose a random number $r \in [0, 1]$.
   (b) If $r < 0.5$, then:
      - Add x_i to z if x_i is non-zero.
   (c) Otherwise:
      - Add y_i to z if y_i is non-zero.
3. Return z.

3.5.5 Mutation

Mutation is a rare event that helps in diversifying our selected population as it introduces random changes to the Y-sparse vectors to promote diversity in the population.

Input: Binary string $a = (a_1, \ldots, a_\ell)$ of length ℓ bits, mutation probability $p_m = \frac{1}{\ell}$.

Output: Mutated binary string a' of length ℓ bits.

$$\Pr(a \xrightarrow{\text{mutation}} a') = p_m^{\text{Ham}(a,a')}(1-p_m)^{\ell-\text{Ham}(a,a')} > 0$$

3.5.6 Encoding and Decoding

The encoding process converts a Y-sparse vector into a bit string representation, encoding each non-zero component as a binary string while preserving the sparsity structure. The decoding process reverses this operation by transforming bit strings back into Y-sparse vectors, allowing evaluation and selection during the algorithm. The procedures are described below:

Encoding Procedure

Input: An integer vector $y = (y_1, \ldots, y_n)$.

Output: A bit string a of length $\ell = \sum_{i=1}^{n} \ell_i$.

1. Compute
$$a = \sum_{i=1}^{n} \text{sgn}(y_i) \cdot 2^{\ell_i - 1}.$$
2. Return a.

Decoding Procedure

Input: A bit string vector $a = (a_1, \ldots, a_\ell)$ of length $\ell = \sum_{i=1}^{n} \ell_i$.

Output: An integer vector $y = (y_1, \ldots, y_n)$.

1. Initialize $\lambda = 1$.

2. For $i = 1$ to n, perform:

   (a) Compute
   $$y_i = \sum_{j=1}^{\ell_i - 1} a_{\lambda + j} \cdot 2^{j-1}$$
   to convert the bit substring back to a decimal integer.

   (b) If $a_{\lambda+\ell_i} = 1$, set $y_i \leftarrow -y_i$ to apply the sign.

   (c) Update $\lambda \leftarrow \lambda + \ell_i$.

3. Return $y = (y_1, \ldots, y_n)$.

3.5.7 Local Search

In the context of the genetic algorithm for the Shortest Vector Problem, local search is a heuristic technique used to improve the quality of solutions by taking into consideration the neighborhood of candidates. It involves examining nearby solutions in multiple iterations to the current vector and moving towards a locally optimal solution.

3.5.8 Heuristic Pruning

Heuristic pruning is another optimization technique employed in the genetic algorithm to enhance efficiency and effectiveness. It involves selectively reducing the search space by eliminating unpromising regions or solutions based on heuristic criteria.

Technique 1:

$$y_i = 0, \quad \text{for } 1 \leq i \leq \lfloor n/2 \rfloor$$

Technique 2:

$$\alpha_i = \sqrt{\frac{\|b_1^*\|}{\|b_i^*\|}}, \quad \text{for } \lfloor n/2 \rfloor < i \leq n$$

3.6 NATURAL NUMBER REPRESENTATION

Natural Number Representation (NNR) is a technique proposed by Fukase and Kashiwabara in 2015 as a key component of their heuristic algorithm for solving the Shortest Vector Problem (SVP). In this representation, each lattice vector is encoded as an array of natural numbers. This encoding is particularly powerful because it establishes a bijection between the vectors in the lattice and their natural number representations, ensuring a one-to-one correspondence. This property is crucial for accurately defining the search space in evolutionary approaches like genetic algorithms. In our genetic algorithm for SVP, we adopt NNR to serve as the chromosome representation of lattice vectors, leveraging its simplicity and structural benefits to enhance the efficiency and clarity of the search process.

3.6.1 Introduction

Given a lattice basis $\mathbf{B}$ and a lattice vector $\mathbf{v} = \sum_{j=1}^{n} u_j \mathbf{b}_j^*$, the natural number representation of $\mathbf{v}$ is a natural number vector

$$\mathbf{z}(\mathbf{v}) = (z_1, z_2, \ldots, z_n) \in \mathbb{N}^n,$$

such that for each j, the coefficient u_j belongs to

$$\left(\frac{-z_j+1}{2}, \frac{-z_j}{2}\right] \quad \text{or} \quad \left[\frac{z_j}{2}, \frac{z_j+1}{2}\right).$$

For a lattice vector $\mathbf{v}$ represented in the form of $\mathbf{v} = \mathbf{B} \cdot \mathbf{u}$, the essence of natural number representation is mapping the rational vector $\mathbf{u} \in \mathbb{Q}^n$ to a natural number vector $\mathbf{z} \in \mathbb{Z}^n$ following some rounding-off rules.

3.6.2 Initialization

To guide the initialization and sampling of candidate vectors, a restricted candidate space $Z(\mathbf{t})$ is defined based on a control vector $\mathbf{t} \in \mathbb{N}^{c+1}$, where $0 = t_0 < t_1 < \cdots < t_c = n$. This restriction reflects the empirical observation that short lattice vectors often have smaller coefficients in the head region and larger ones toward the tail.

Given a vector $\mathbf{t} \in \mathbb{N}^{c+1}$, define the set

$$Z(\mathbf{t}) = \left\{ \mathbf{z} \in \mathbb{N}^n : \text{for } t_j < i \leq t_{j+1},\ 0 \leq z_i \leq j \right\},$$

where $j = 0, 1, \ldots, c-1$.

This definition ensures that each component z_i of a candidate vector is bounded by a value depending on its position i, partitioned by the intervals defined in $\mathbf{t}$.

One practical way to implement this is by constructing a *template vector* $\bar{\mathbf{z}} \in \mathbb{Z}^n$, where for each interval $t_j < i \leq t_{j+1}$, we assign $\bar{z}_i = j$. For instance, choosing $\mathbf{t} = [0, 20, 25, 30]$ for a 30-dimensional vector gives the template vector:

$$\bar{\mathbf{z}} = [\underbrace{0, \ldots, 0}_{20}, \underbrace{1, \ldots, 1}_{5}, \underbrace{2, \ldots, 2}_{5}].$$

Then, a candidate NNR vector $\mathbf{z}$ belongs to $Z(\mathbf{t})$ if and only if it satisfies:

$$z_i \leq \bar{z}_i \quad \text{for all } i = 1, \ldots, n.$$

Input:

- $\mathbf{t} = (t_0, t_1, \ldots, t_c) \in \mathbb{N}^{c+1}$, the parameter vector defining the candidate NNR set $Z(\mathbf{t})$.
- $N_{\text{population}}$, the desired size of the initial population.

Procedure:

1. Construct the template vector $\bar{\mathbf{z}} \in \mathbb{Z}^n$ such that for each interval $t_j < i \leq t_{j+1}$, set $\bar{z}_i = j$.

2. Initialize the parent population set as empty: $S_{\text{parent}} \leftarrow \emptyset$.

3. Repeat until $|S_{\text{parent}}| = N_{\text{population}}$:

   (a) Generate a vector $\mathbf{z} = (z_1, z_2, \ldots, z_n) \in \mathbb{N}^n$ where each z_i is sampled uniformly at random from the interval $[0, \bar{z}_i]$.

   (b) Add $\mathbf{z}$ to the parent set: $S_{\text{parent}} \leftarrow S_{\text{parent}} \cup \{\mathbf{z}\}$.

Output: S_{parent}, a set of $N_{\text{population}}$ NNR vectors belonging to $Z(\mathbf{t})$.

3.6.3 Crossover

The crossover operator combines genetic material from two parent chromosomes to generate a new offspring, aiming to inherit and mix beneficial traits from both parents. In this method, two parents are selected uniformly at random from the current population. The idea is to preserve some initial structure in the offspring by setting the first few components to zero (corresponding to the early region of an NNR vector), while the remaining components are inherited from either parent with equal probability.

Procedure:

1. Select two parent vectors $\mathbf{z}_a$, $\mathbf{z}_b$ uniformly at random from S_{parent}.

2. Set $z_{\text{offspring}}[i] := 0$ for all $i = 1, \ldots, t[1]$.

3. For $i = t[1] + 1, \ldots, n$, set:

$$z_{\text{offspring}}[i] := \begin{cases} z_a[i], & \text{with probability } 0.5, \\ z_b[i], & \text{with probability } 0.5. \end{cases}$$

Output: An offspring chromosome $\mathbf{z}_{\text{offspring}}$ composed of a fixed initial region and mixed inherited traits from both parents.

3.6.4 Mutation

Mutation is a rare but essential genetic operator in evolutionary algorithms. It introduces small, random changes to the offspring chromosomes to enhance the diversity of the population and prevent premature convergence to local optima.With a certain mutation probability p_{mutation}, each mutable gene has a chance to be modified by adding a discretized Gaussian noise.

Input:

- $S_{\text{offspring}}[k]$: an offspring chromosome from crossover.
- p_{mutation}: mutation probability.

Procedure:

1. For each gene index $i = t[1]+1,\ldots,n$, do:
   (a) Generate a random number $r \in [0,1]$.
   (b) If $r < p_{\text{mutation}}$, then:
      - Sample $r_{\text{snd}} \sim \mathcal{N}(0,1)$, a standard Gaussian variable.
      - Round r_{snd} toward infinity using $\text{RI}(r_{\text{snd}}) := \text{sign}(r_{\text{snd}}) \cdot \lceil |r_{\text{snd}}| \rceil$.
      - Update:

$$z_{\text{offspring}}[i] := \max\left(z_{\text{offspring}}[i],\ z_{\text{offspring}}[i] + \text{RI}(r_{\text{snd}})\right)$$

   (c) Else:
      - Leave $z_{\text{offspring}}[i]$ unchanged.

Output: A mutated chromosome $z_{\text{offspring}}$, potentially improved in diversity through controlled random alterations.

3.6.5 Fitness Evaluation Function

The fitness function plays a vital role in guiding the search process of the genetic algorithm. For each individual chromosome $\mathbf{z} \in \mathbb{N}^n$, a fitness value $f(\mathbf{z})$ is computed to evaluate how close the corresponding lattice vector is to the shortest vector in the lattice. This helps in selecting the best candidates for reproduction and survival in the next generation.

Input:

- $\mathbf{z}$: a chromosome vector in $\mathbb{N}^n$
- $\mathbf{B} = [\mathbf{b}_1, \ldots, \mathbf{b}_n]$: the lattice basis

Output:

- $f(\mathbf{z}) = \|\mathbf{v}\|^2$: the squared norm of the lattice vector $\mathbf{v}$ corresponding to $\mathbf{z}$ under the lattice basis $\mathbf{B}$

Procedure:

1. Compute the Gram–Schmidt orthogonalized basis $\mathbf{B}^*$ and the transformation matrix $\mathbf{U}$, where $\mathbf{B} = \mathbf{U}(\mathbf{B}^*)^\top$ and $\mathbf{U}(i, j) = \mu_{i,j}$.
2. Initialize a vector $\mathbf{u} \in \mathbb{R}^n$ to zero.
3. For $i \leftarrow n$ down to 1, do:
   (a) Compute
   $$y = -\sum_{j=i+1}^{n} u_j \mu_{j,i}$$
   (b) Update $u_i \leftarrow \lfloor y + 0.5 \rfloor$
   (c) If $u_i \leq y$, then
   $$u_i \leftarrow u_i - (-1)^{z_i} \left\lfloor \frac{z_i}{2} \right\rfloor$$

(d) Else,

$$u_i \leftarrow u_i + (-1)^{z_i} \left\lfloor \frac{z_i}{2} \right\rfloor$$

4. Compute the fitness value:

$$f(\mathbf{z}) = \sum_{i=1}^{n} |u_i|^2 \|\mathbf{b}_i^*\|^2 = \|\mathbf{v}\|^2$$

Importance: This fitness evaluation determines the quality of each candidate vector $\mathbf{z}$. A lower value of $f(\mathbf{z})$ indicates a shorter corresponding lattice vector $\mathbf{v}$, hence a better approximation to the solution of the Shortest Vector Problem (SVP). By favoring individuals with lower fitness values, the genetic algorithm effectively guides the population toward optimal or near-optimal solutions.

3.6.6 Elitist Selection Strategy

To ensure that the best solutions are not lost across generations, an *elitist strategy* is employed during the selection process. This strategy guarantees that a fixed number of top-performing chromosomes based on their fitness values will always be preserved in the population.

Motivation: Evolutionary algorithms are stochastic in nature, and without an elitist strategy, high-quality solutions from one generation can be lost due to random fluctuations. Elitism safeguards against this by preserving the most promising individuals.

Procedure:

1. Let S_{parent} be the current generation of chromosomes, and $S_{\text{offspring}}$ be the newly created generation from crossover and mutation.

2. Combine the two sets:

$$S_{\text{combined}} = S_{\text{parent}} \cup S_{\text{offspring}}.$$

3. Sort all individuals in S_{combined} in ascending order based on their fitness values.

4. Select the top $N_{\text{population}}$ chromosomes from the sorted list to form the next generation S_{next}.

Output: A new population S_{next} of size $N_{\text{population}}$, which includes the fittest chromosomes from both the current and previous generations.

Advantages:

- Prevents the loss of the best-found solutions.
- Improves convergence speed by retaining high-quality individuals.
- Balances exploration (via crossover and mutation) with exploitation (via elitist preservation).

3.6.7 Encoding and Decoding

In NNR (Natural Number Representation), lattice vectors are represented using an intermediate natural number vector $\mathbf{z} \in \mathbb{N}^n$, rather than directly using the coefficient vector $\mathbf{x} \in \mathbb{Z}^n$ relative to basis $\mathbf{B}$. This intermediate form enhances compatibility with genetic algorithm operations, which typically work with non-negative integers.

This section describes the two main operations:

- **Encode:** Converts a lattice coefficient vector $\mathbf{x}$ into a natural number vector $\mathbf{z}$.
- **Decode:** Reconstructs the original lattice vector $\mathbf{v} = \mathbf{B} \cdot \mathbf{u}$ from the NNR vector $\mathbf{z}$.

Encode Procedure $(\mathbf{x} \to \mathbf{z})$ **Inputs:** Integer vector $\mathbf{x} \in \mathbb{Z}^n$, Gram–Schmidt coefficients $\mu \in \mathbb{R}^{n \times n}$

Output: Vector $\mathbf{z} \in \mathbb{N}^n$

1. For each $i = 1, \ldots, n$, do:
   (a) Initialize $m_i \leftarrow x_i$.
   (b) For each $j = i+1, \ldots, n$, update:

$$m_i \leftarrow m_i + x_j \cdot \mu_{j,i}.$$

   (c) Set $m_i \leftarrow 2 \cdot m_i$.
   (d) If $m_i > 0$, then:
      i. If m_i is not an integer, set $m_i \leftarrow \lfloor m_i \rfloor$.
      ii. Else, set $m_i \leftarrow m_i - 1$.
      iii. Set $z_i \leftarrow \lfloor m_i \rfloor$.
   (e) Else (if $m_i \leq 0$):
      i. Set $m_i \leftarrow -m_i$.
      ii. Set $z_i \leftarrow \lfloor m_i \rfloor$.

Decode Procedure $(\mathbf{z} \to \mathbf{v})$ **Inputs:** Vector $\mathbf{z} \in \mathbb{N}^n$, Gram–Schmidt coefficients μ, lattice basis $\mathbf{B}$

Output: Lattice vector $\mathbf{v} = \mathbf{B} \cdot \mathbf{u}$

1. Compute the Gram–Schmidt orthogonalized basis $\mathbf{B}^*$ and transformation matrix $\mathbf{U}$, where $\mathbf{B} = \mathbf{U}(\mathbf{B}^*)^\top$ and $\mathbf{U}(i,j) = \mu_{i,j}$.
2. Initialize $\mathbf{u} \in \mathbb{R}^n$ as the zero vector.
3. For $i = n, n-1, \ldots, 1$, do:

(a) Compute

$$y = -\sum_{j=i+1}^{n} u_j \mu_{j,i}.$$

(b) Set

$$u_i \leftarrow \lfloor y + 0.5 \rfloor .$$

(c) If $u_i \leq y$, then

$$u_i \leftarrow u_i - (-1)^{z_i} \left\lfloor \frac{z_i}{2} \right\rfloor .$$

(d) Else

$$u_i \leftarrow u_i + (-1)^{z_i} \left\lfloor \frac{z_i}{2} \right\rfloor .$$

4. Return $\mathbf{v} = \mathbf{B} \cdot \mathbf{u}$.

3.7 COMPARISON: Y-SPARSE VS NNR

In the context of genetic algorithms for lattice problems, choosing an appropriate encoding scheme is critical for maintaining diversity, computational efficiency, and numerical stability. We compare two such representations—Y-sparse and Natural Number Representation (NNR)—based on our experimentation and empirical observations.

Y-Sparse Representation

Y-sparse representation attempts to encode lattice vectors using signed integers, where many entries are zero (i.e., the vector is sparse). While this sparsity can reduce computational overhead, we encountered several challenges in practical implementations:

- **Loss of Diversity:** During evolution, chromosomes containing negative integers often failed to carry over into subsequent generations. This led to a sharp reduction in genetic diversity and convergence to suboptimal solutions.

- **Sign Bit Management:** Representing both positive and negative values required explicit sign handling, which complicated the encoding and increased chromosome length.

- **Floating-Point Precision Issues:** The fitness function in Y-sparse is typically based on the inverse of the vector's squared norm. This introduces floating-point operations, which can lead to rounding errors and instability in selection and ranking.

Natural Number Representation (NNR)

To overcome the limitations of Y-sparse, we adopted a Natural Number Representation, where all gene values in chromosomes are non-negative integers. This transition yielded several advantages:

- **Preserved Diversity:** By eliminating negative numbers, all encoded values remained valid and inheritable. This led to improved diversity and robustness in the population across generations.

- **Simplified Encoding:** The absence of sign bits resulted in shorter chromosomes and simplified genetic operations such as crossover and mutation.

- **Deterministic Fitness Evaluation:** Unlike Y-sparse, NNR uses uniform selection based on integer metrics. This avoids floating-point errors and increases computational precision and efficiency.

Conclusion

Overall, NNR provides a more stable and efficient foundation for genetic algorithms in lattice-based problems. It not only maintains population diversity but also enhances compatibility with evolutionary operations by operating purely over

non-negative integers. The shift from Y-sparse to NNR significantly improved the exploratory capability and convergence behavior of our algorithm.

Chapter 4

METHODOLOGY

This chapter discusses the methods used throughout the project. We have implemented the genetic algorithm in C++. Our primary goal was to replicate the algorithm and generate solutions for as many dimensions as possible.We have tested our code using the lattices from SVPchallenge website mentioned on our github.

Our approach involves Multiple classes that deal with different sections of the code and interact with each other to achieve the desired functionality such that the overall is highly modular and loosely coupled. All of our classes are defined in terms of templates and can be instantiated as per the user's requirement of how precise results are needed. We will now explain how each of these components work and interact with each other.

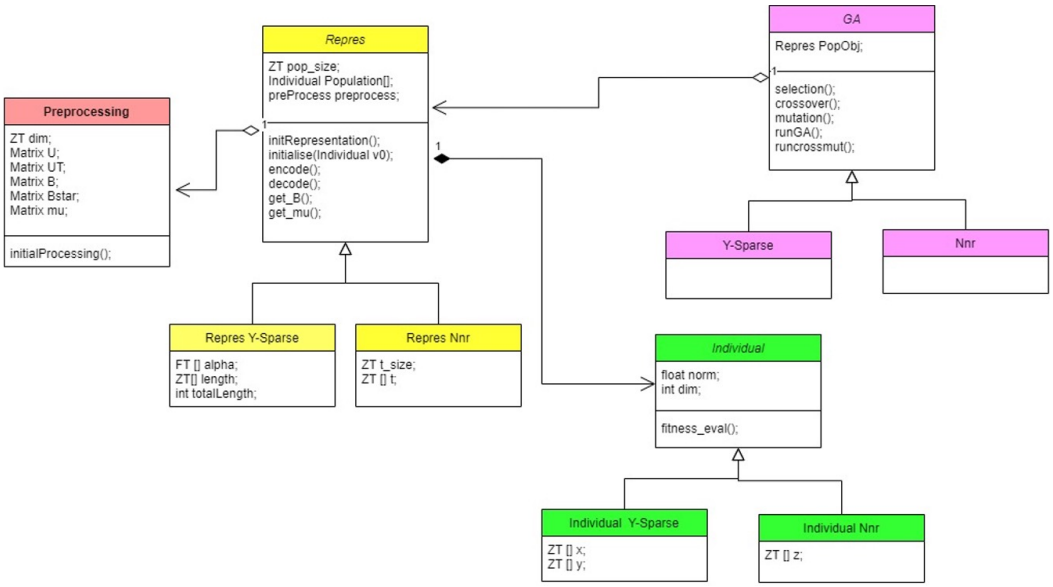


Figure 4.1: Class Diagram

4.1 PREPROCESS CLASS

This class gives us the functionality to take input of these lattice based matrices and perform a variety of operations on it. We use generic data types ZT(integer) and FT(float) for flexibility. The operations run on the input are summarised as follows:

- **Input Matrix Handling:** Reads the lattice basis matrix from a file or generate new lattice basis based on the best vectors obtained in the previous iteration of the genetic algorithm.
- **BKZ/LLL Reduction:** Apply lattice reduction algorithms (BKZ or LLL) to transform the basis into a reduced form this gives us a set of better basis to start with. Typically we select a block size of about $\sqrt{n}$ for the BKZ reduction.
- **Gram-Schmidt Orthogonalization (GSO):** Compute the Gram-Schmidt basis (B^*) and the GSO coefficients (μ) for the reduced matrix, which are essential for lattice algorithms.
- **Output and Storage:** Writes the reduced basis and intermediate matrices to files and initializes dynamically allocated structures for further computations.

4.2 INDIVIDUAL CLASS

An object of the Individual Class represents an "Individual" in the population and has its member variables as the corresponding representations of the individual along with its norm, which is very essential in calculating the individual's fitness to participate in crossover. It also has some utility functions that are required for crossover and mutation and the get norm function that returns the norm of the individual. Major functionalities covered by this class are as follows:

- **Initialization:** It construct the corresponding representation of the individual by sampling the vector in a restricted search space.
- **Fitness Evaluation:** It computes the individual's fitness by computing the norm corresponding to the vector represented by our individual.
- **Utility Functions:** The class includes certain utility functions that are required for crossover and mutation.

The `Individual` class is extended and some of it's methods are overridden based on the representation scheme used:

- **Individual Y-Sparse:** Each object stores both the coordinate vector x and its corresponding y-sparse encoding.
- **Individual NNR:** Each object represents the individual solely in terms of the natural number vector z.

4.3 REPRES CLASS

The repres class is used to hold an entire generation of our population and it does so by storing the entire population in an array called population, It also contains an object of the pre-processing class, the population size, and the dimension of the lattice as its member variables.The repres class also has an initialize function that initializes our population Along with initialize it and other member functions such as encode and decode which help in the encoding and decoding of our individuals. Some of the core features of this class are as follows:

- **Initialization:** It construct the initial population generation for our Algorithm based on a certain pre-processing object.
- **Encoding And Decoding:**The class also provides functions to decode the encoded individual and reconstruct its original lattice vector representation.

This class hence allows us to use various different types of representation formats for our GA, without inducing any dependence on any other class.

The `repres` class is extended and some of it's methods such are overridden based on the representation scheme used:

- **Repres Y-Sparse:**Stores the α vector and the corresponding length vector used for initialization.
- **Repres NNR:**Stores the template vector used for initialization.

4.4 GA CLASS

The GA class is an abstract class that acts as interface for the implementation of Genetic Algorithm.It has an object of the repres class as it's data members that is used by the class to refer to the population of individuals.It also contains many virtual functions such as selection,crossover,mutation and runGA that can be implemented as per the chosen methodologies for the genetic algorithm.The class also has a compare function that compares two individuals based on their norm and returns a boolean value regarding which has greater norm,this is used in sorting the population based upon the norm of the individuals.

The `GA` class is extended and some of it's methods such are overridden based on the representation scheme used:

- **Y-Sparse**

  The Y-Sparse class is an implementation of the GA class and overloads all the virtual functions of the GA class as per the methodologies mentioned in [1]. Also, some additional utility functions are defined such as logical and and logical xor that help in performing the "xor" and "and" of the chromosomes bit-string during crossover and mutation.

This Y-Sparse_ls class further extends the Y-Sparse class and overloads the runcrossmut function to add local search, making the overall algorithm more efficient in obtaining results optimally.

- **NNR**

  The NNR class is an implementation of the GA class and overloads all the virtual functions of the GA class as per the methodologies mentioned in [2] and also defines some utlitly functions to aid the crossover and mutation functions.

  This NNR_ls class further extends the NNR class and overloads the runcrossmut function to add local search, making the overall algorithm more efficient in obtaining results optimally.

4.5 NTRU CLASS

We have also implemented an NTRU class to test our algorithm on special lattices called the NTRU lattices. We implement a code to generate the basis for this lattice following the sampling procedures for NTRU cryptosystems. It involves polynomial operations, modular arithmetic, and sampling methods to prepare the necessary components for encryption, decryption, and key generation.

Chapter 5

UNIQUE ASPECTS OF THE PROJECT

5.1 APPICATION OF OOPS

The importance of the work is not merely in generating output for the given problem but also in its ability to be replicated and used for different implementations and constructs of the genetic algorithm. Here are various parts where the OOP principles are highlighted:

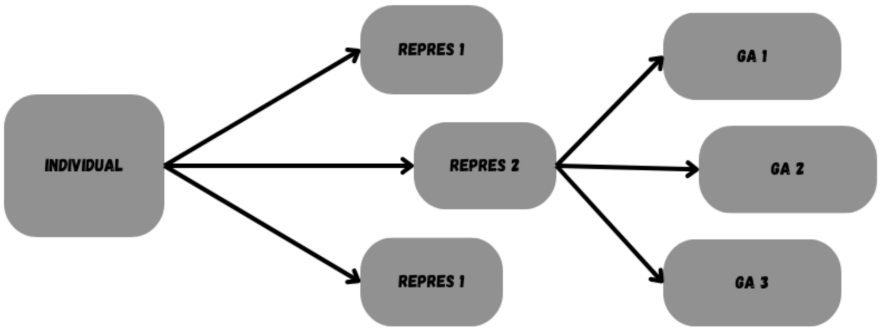


Figure 5.1: Example Usage of OOPs

- **(a)** Firstly, we can talk about how the `Individual` class can be used for various types of replications of the code, as it can be used for multiple replications since the encoding and decoding are handled by the `repres` class.
- **(b)** The `repres` class embodies the concept of Y-sparse representation. Had the algorithm been different from the genetic algorithm, the same `repres` object, named `PopObj`, could have been used in other implementations as

well. Another point to add is that any class other than `repres` that provides a new `PopObj` would also work seamlessly.

- The `GA` class is inherited, allowing offspring to provide specific and different implementations as required. This class is also abstract, providing a structure that can be further modified by inheriting classes.

5.2 USE OF `FPLLL` LIBRARY FOR EFFICIENT LATTICE COMPUTATIONS

In our project, we utilize the `fplll` library to achieve faster computations with higher numerical precision. Our code templates employ the `ZT` (integer) and `FT` (floating-point) types to allow generality across different numeric representations.

The `fplll` library implements a variety of lattice algorithms based on floating-point orthogonalization techniques. At its core lies the LLL (Lenstra–Lenstra–Lovász) lattice basis reduction algorithm—hence the name `fplll`. The library offers several floating-point LLL variants, each balancing speed and mathematical guarantees differently. To optimize performance, `fplll` provides a "wrapper" that automatically selects the best sequence of these algorithm variants, delivering guaranteed correctness with minimal runtime. This wrapper abstracts away the complexity from the user.

Moreover, `fplll` seamlessly integrates with the MPFR (Multiple Precision Floating-Point Reliable) and GMP (GNU Multiple Precision Arithmetic) libraries, enabling robust support for arbitrary precision arithmetic when required, which is essential for high-precision lattice computations.

5.2.1 Key Features of fplll

- **Lattice Algorithms:** `fplll` provides implementations of several lattice algorithms, focusing on efficiency and accuracy. These algorithms are fundamental in areas such as cryptography, cryptanalysis, and optimization.

- **Floating-Point LLL Reduction:** The library includes implementations of floating-point LLL reduction algorithms, which are essential for lattice reduction. Lattice reduction aims to find a ”nice” basis for a lattice, often used to simplify lattice-based cryptographic schemes or to solve lattice problems efficiently. `fplll` offers algorithms with different trade-offs between speed and guarantees, enabling high performance and accuracy in lattice computations.

- **Wrapper for Algorithm Selection:** A wrapper in `fplll` automatically selects the best sequence of algorithm variants based on estimates. This wrapper aims to provide guaranteed output as quickly as possible, abstracting the complexities of algorithm selection from the user, making `fplll` easier to use without deep knowledge of the underlying algorithms.

- **Integration with MPFR and GMP Libraries:** `fplll` integrates with MPFR and GMP libraries, providing high-performance arbitrary-precision arithmetic. These libraries are crucial for ensuring accurate and efficient floating-point computations, thus enhancing the usability and robustness of `fplll` in lattice computations.

Overall, `fplll` is a comprehensive library for lattice computations, offering a range of lattice algorithms, optimization techniques, and integration with key mathematical libraries. Its emphasis on efficiency, precision, and ease of use makes it a valuable tool for researchers and practitioners in fields like cryptography, optimization, and computational mathematics.

5.2.2 Documentation Source

We have followed the official documentation for `fplll` to understand its functionality and integrate it into our project. The documentation is available at: `fplll/fplll: Lattice algorithms using floating-point arithmetic` on GitHub.

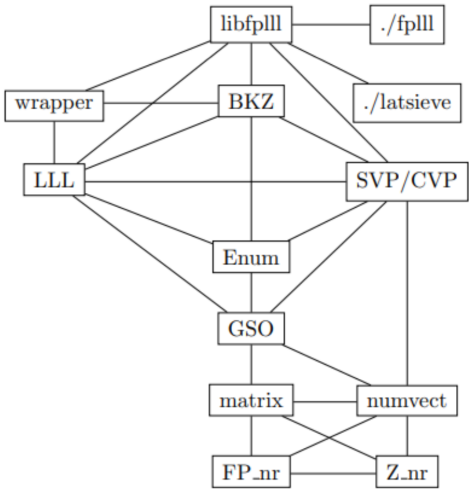


Figure 5.2: FPLLL Structure

5.3 ENHANCING FLEXIBILITY WITH TEMPLATES

Our implementation heavily relies on C++ templates which have given us a smart and efficient to handle different data types without sacrificing performance. This makes our code reusable and gives it type safety.

5.3.1 Template-based Classes in Our Project

In our project, template classes such as `Individual`, `repres`, and `Preprocessing` enable the generic handling of different data types, operations, and optimization methods used in the genetic algorithm. The key operations (e.g., matrix multiplication, Gram-Schmidt orthogonalization, and encoding/decoding) are abstracted in template-based methods, making the codebase both flexible and maintainable.

5.4 MEMORY MANAGEMENT WITH SMART POINTERS

Memory Management is critical whenever we are dealing with dynamically allocated memory.We faced issues with memory leaks where dynamically allocated memory was not being freed properly leading the application to consume increasing amounts of memory overtime. Due to this reason we were unable to run our code for higher dimension lattices.

We had to trace the pointers and ensure each allocation had a corresponding de-allocation. But still the code was too complex to single out the area of error. Thus we resorted to the use of smart pointers which gave us the following features:

- **Automatic Memory Management:** Smart pointers automatically release the memory they point to when they go out of scope. This reduces the risk of memory leaks and ensures that memory is freed as soon as it is no longer needed.
- **Simplified Code:** By eliminating the need for manual memory management we need not call functions like 'delete' and 'free'.
- **Enhanced Safety:** Unique pointers and shared pointers provide safety by preventing accidental memory access after it has been freed.

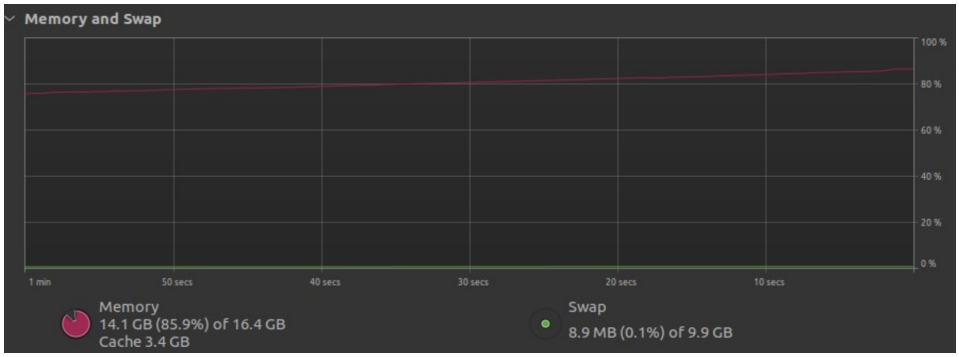


Figure 5.3: Memory usage without Smart Pointers

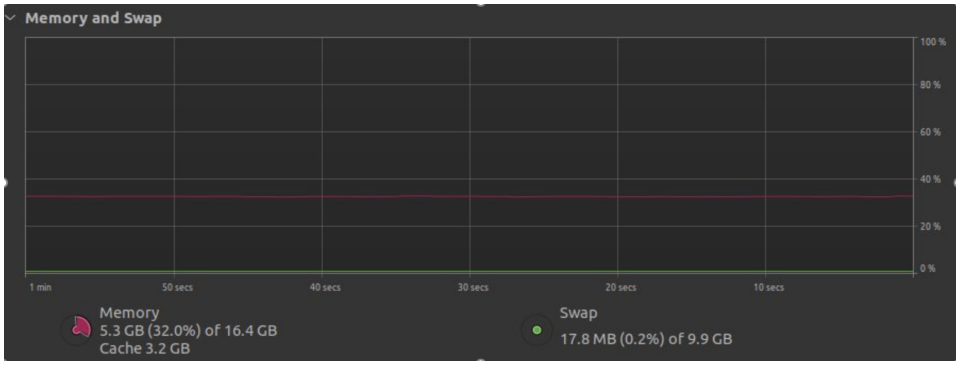


Figure 5.4: Optimized memory usage

Chapter 6

RESULTS & DISCUSSION

We tested our code for smaller dimension lattices, ranging from 40 to 80. To find the true value (close approximation) of the shortest vector in these lattices we applied the BKZ algorithm on these lattices with block size of 40. Given below is the comparision of our results using the Y-sparse representation with the ones obtained from the deterministic algorithm.

Dimension	**Genetic Algorithm + Local Search**	**Time Taken (s)**	**BKZ Algorithm (Beta = 40)**
40	1702	1.1	1702
42	1553	1.5	1553
44	1680	22.62	1680
46	1728	311.32	1728
48	1803	805.87	1822
50	1902	422.23	1893
52	1915	143	1915

Table 6.1: Comparison of GA with Y sparse + Local Search and BKZ Algorithm (Beta = 40) Across Different Dimensions.

We were able to successfully identify short vectors in lattices of dimensions up to 52 using the Y-sparse representation. This approach proved effective for lower to mid-range dimensions; however, as the dimensionality increased, the computational effort and time required grew significantly, eventually limiting its applicability beyond the 52-dimensional threshold.

To explore alternatives, we then evaluated the same set of lattices using the Natural Number representation. This method demonstrated a clear advantage in terms of computational efficiency, significantly reducing the time required to

identify short vectors when compared to the Y-sparse approach. Moreover, the Natural Number representation enabled us to extend our analysis beyond the 52-dimensional limit imposed by the Y-sparse representation. As a result, it proved to be a more scalable and effective representation for handling high-dimensional lattice problems.

Dimension	**Genetic Algorithm + Local Search**	**Time Taken (s)**	**BKZ Algorithm (Beta = 40)**
40	1702	0.064	1702
42	1553	0.233	1553
44	1680	0.511	1680
46	1728	0.422	1728
48	1803	2.98	1822
50	1902	3.26	1893
52	1915	2.6	1915
54	1921	41.6	1921
56	2003	40.5	1973
58	2038	45	2038
60	1956	190	1956
62	2125	135	2111
64	2109	167	2103
68	2152	421	2141
72	2278	3751	2179
76	2318	53713	2240
80	2380	148137	2429
84	2551	341348	2421
88	2698	12922	2490

Table 6.2: Comparison of GA with NNR + Local Search and BKZ Algorithm (Beta = 40) Across Different Dimensions.

The graph presented below provides a comparative visualization of the time taken by the two different representations—Y-sparse and Natural Number Representation—when applied to the same set of lattice dimensions. The time values are plotted on a logarithmic scale to accommodate the wide range of magnitudes and to highlight performance differences more clearly, especially in higher-dimensional

lattices. This logarithmic scaling allows for better interpretation of exponential trends and subtle variations in execution time between the two methods, offering valuable insights into their relative computational efficiencies.

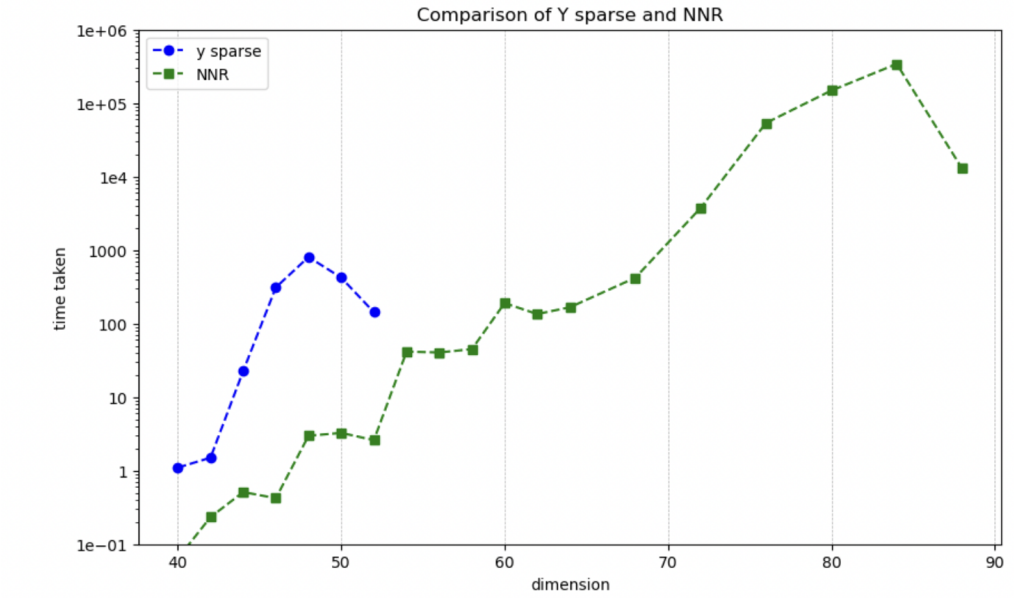


Figure 6.1: Comparison of Y-sparse and NNR

Chapter 7

FUTURE WORKS

- Investigate Genetic Algorithm-based attacks on the **NTRU cryptosystem** by leveraging structural patterns extracted from shortest vectors encoded with the Natural Number Representation (NNR). These patterns can be used to **intelligently initialize** and guide the algorithm's population, potentially improving convergence speed and accuracy in recovering private keys.
- Improve the runtime of the Genetic Algorithm by introducing **GPU-based parallel computing**.
- Explore other evolutionary algorithms, such as **Particle Swarm Optimization (PSO)**:
  - Conduct comparisons with the Genetic Algorithm to assess performance on similar tasks.
  - Identify opportunities for hybrid approaches that combine the strengths of both algorithms.

BIBLIOGRAPHY

[1] Dan Ding, Guizhen Zhu, and Xiaoyun Wang. A genetic algorithm for searching the shortest lattice vector of svp challenge. In *Proceedings of the 2015 annual conference on genetic and evolutionary computation*, pages 823–830, 2015.

[2] Masaharu Fukase and Masahiro Kaminaga. Improving genetic algorithms for solving the svp: focusing on low memory consumption and high reproducibility. *Iran Journal of Computer Science*, 5(4):359–372, 2022.

[3] Masaharu Fukase and Kenji Kashiwabara. An accelerated algorithm for solving svp based on statistical analysis. *Journal of Information Processing*, 23(1):67–80, 2015.

[4] Luan Luan, Chunxiang Gu, and Yonghui Zheng. A genetic algorithm with restart strategy for solving approximate shortest vector problem. In *2020 12th International Conference on Advanced Computational Intelligence (ICACI)*, pages 243–250. IEEE, 2020.

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

100 Not Cited or Quoted 12%
Matches with neither in-text citation nor quotation marks

10 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

9% Internet sources
10% Publications
2% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.